MES College of Arts, Commerce and Science

Department of Computer Applications

Project Synopsis

1. Title of the Project Sentinel X – AI-Powered Cybersecurity and Threat Mitigation System

2. Name of the Guide [Insert Guide Name Here]

3. Name of Team Members

1. Namith MR | UUCMS: U18EM24S0036 | Phone: 9019974413
2. Jagadeesh N | UUCMS: U18EM24S0142 | Phone: 6363854860
3. Balaji R | UUCMS: U18EM24S0054 | Phone: 8152052771

4. Start Date and Tentative End of the Project

- **Start Date:** 01-08-2026
- **Tentative End Date:** 12-10-2026

5. Hardware and Software Requirements

- **Hardware Requirements:**
  - Processor: Intel Core i5 / AMD Ryzen 5 (or equivalent quad-core processor).
  - RAM: 8 GB minimum (16 GB required if running the local LLM Expert System concurrently).
  - Storage: 256 GB SSD (to store local AI model weights and database logs).
  - GPU: Integrated Graphics are sufficient for basic prototype execution (A dedicated entry-level GPU like NVIDIA GTX 1650 is optional but recommended for faster deepfake detection).

    - Peripherals: None required for development. (WebAuthn biometrics can be simulated using Chrome DevTools Virtual Authenticators or demonstrated using a smartphone via cross-device QR code).
- **Software Requirements:**
    - **Frontend Development:** React.js / HTML5, CSS3, Tailwind CSS.
    - **Backend Development:** Python 3.10+, FastAPI (for asynchronous API routing).
    - **Machine Learning & AI:** PyTorch (Computer Vision), Scikit-learn (URL Classification), Local LLM API (e.g., Ollama/Llama-3).
    - **Database & Security:** PostgreSQL, Python `hashlib` (SHA-256 for audit logging), WebAuthn API.
    - **IDE & Tools:** VS Code, Git/GitHub for version control, Postman for API testing.

6. Problem Statement As digital threats evolve, standard consumer-level cybersecurity is failing to keep pace with AI-generated attacks. Users face highly sophisticated phishing URLs and hyper-realistic deepfake images that bypass traditional filters. Furthermore, when a threat is detected by current systems, non-technical users are often left with a generic warning and no actionable guidance on how to secure their systems. Additionally, standard security logs are vulnerable to tampering by malware, making post-incident analysis unreliable. There is a critical need for a unified system that not only detects modern AI-driven threats but also guides the user through mitigation while keeping system records mathematically verifiable.

7. Existing System

- **Fragmented Tools:** Users currently rely on separate, disconnected applications for antivirus, password management, and web filtering.
- **Signature-Based Detection:** Existing URL scanners rely heavily on static blocklists, which fail to detect newly generated (zero-day) phishing domains.
- **Lack of Deepfake Detection:** Standard consumer security suites do not offer image authentication or synthetic media detection.
- **Unprotected Logs:** Security logs in existing personal systems are stored in plain text or standard databases, making them easily deletable or alterable by unauthorized users or malware.

- **No Mitigation Guidance:** Existing systems act merely as alarms. They alert the user of a breach but provide no contextual, step-by-step resolution strategy.

8. Proposed System and Objective Proposed System: Sentinel X is a comprehensive, closed-loop cybersecurity application that integrates Machine Learning, Deep Learning, Cryptography, and Generative AI into a single platform. The system uses advanced predictive models to detect threats rather than relying on static lists, and it pairs threat detection with an AI-driven Expert System to provide immediate mitigation strategies.

Objectives:

- To develop a Machine Learning model (Random Forest/XGBoost) capable of analyzing URL features to detect phishing sites in real-time.
- To implement a PyTorch-based Deep Learning vision model for classifying images as real or deepfake (synthetic).
- To calculate and display a real-time "Device Security Score" based on system posture and password integrity checks (utilizing k-Anonymity via the Have I Been Pwned API).
- To integrate a local Large Language Model (LLM) acting as an Expert System Chatbot, which dynamically generates tailored recovery steps when a threat is flagged.
- To secure all system events and threat detections using a lightweight Cryptographic Hash-Chain (SHA-256), creating a tamper-proof audit log.
- To secure user access via the WebAuthn API for hardware-backed biometric authentication (fingerprint/facial recognition).

9. Methodology The project will be developed using an iterative, modular approach, divided across three parallel domains to facilitate efficient team collaboration:

- **Step 1: Machine Learning & AI Modeling (Module 1)**
  - Data Collection: Acquire datasets for phishing URLs (e.g., PhishTank) and deepfake images.
  - Model Training: Extract URL parameters (entropy, length, IP presence) and train a classification model. Fine-tune a lightweight CNN (e.g., ResNet-18) for deepfake detection. Export models for backend integration.
- **Step 2: Core Backend & Logic Construction (Module 2)**

- Establish a FastAPI server to handle asynchronous requests.
- Implement the cryptography module utilizing SHA-256 to create an append-only, immutable database structure for security logs.
- Integrate the AI models into the API routes and construct the context-aware prompt engine for the LLM Expert System.

- **Step 3: Frontend Integration & Security Interfaces (Module 3)**
  - Design the user dashboard using React.js/Tailwind.
  - Implement the WebAuthn API for biometric login flows.
  - Connect the frontend interface to the FastAPI backend, establishing real-time data visualization (Security Score gauges, Threat chat interfaces).
- **Step 4: System Testing and Deployment**
  - Conduct unit testing on individual AI models and integration testing on the complete API pipeline.

System Information Flowchart (Methodology Support) *(Note for the report: You can draw this flowchart using blocks and arrows in MS Word based on this structure)*

```
[User Authentication] -> (WebAuthn Biometrics)
         |
         v
[User Dashboard Input] -> (URL String / Image File / Password)
         |
         v
[FastAPI Gateway] ---> [Threat Analysis Engines]
                              |-- ML URL Classifier
                              |-- PyTorch Deepfake Detector
                              |-- Password Entropy Checker
         |
         v
[Detection Results & Risk Scoring]
```

```
         |
         +--> If Threat == TRUE:
         |      v
         |    [AI Expert System / LLM] -> (Generates Mitigation Steps)
         |
         +--> [Cryptographic Hash-Chain Engine] -> (Secures log via SHA-256)
         |
         v
[Database Storage] -> (Tamper-proof Ledger)
         |
         v
[Frontend Dashboard] -> (Displays Risk Score, AI Advice, and Audit Log)
```